

Apellidos:

Nombre:

---

## Concurrencia y Paralelismo

### Bloque I: Concurrencia

Junio 2023

#### 1. Barrera [2.5 puntos]

Una barrera es una estructura de sincronización que para a un grupo de threads hasta que un cierto número ha llegado hasta ella. Los distintos threads llaman a una función `barrier()` que marca el punto de espera:

---

```
// N Threads corriendo en este código
...
barrier(b); // Todos los threads paran aquí hasta que n lleguen a este punto
...
```

---

Defina la estructura `barrier_t` y la función `barrier` de forma que paren a los threads hasta que `n` hayan llegado a la barrera.

---

```
typedef struct {
    int n; // Número de procesos que tienen que llegar a la barrera para que continuen.
    ...
} barrier_t;

void barrier(barrier_t *b) {
    ...
}
```

---

```
int mtx_lock(mtx_t *mtx);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
int cnd_signal(cnd_t *cond);
int cnd_broadcast(cnd_t *cond);
```

---

#### Solución:

---

```
typedef struct {
    int n; // Número de procesos que tienen que llegar a la barrera para que continuen.
    int thr_reached; // starts at 0
    mtx_t lock;
    cnd_t barrier_wait;
} barrier_t;

void barrier(barrier_t *b) {
    mtx_lock(&b->lock);
    b->thr_reached++;
    if(b->thr_reached < b->n)
        cnd_wait(&b->barrier_wait, &b->lock);
    else
        cnd_broadcast(&b->barrier_wait);
    mtx_unlock(&b->lock);
}
```

---

## 2. Paso de Peatones [2.5 puntos]

Vamos a simular un paso peatonal con un semaforo que regula el paso tanto de vehiculos como de peatones. El semaforo, cada peatón y cada coche está representado por un thread. La variable green indica quien tiene el semaforo en verde.

Los peatones pueden cruzar todos a la vez, pero los vehiculos tienen que ir de uno en uno respetando su orden de llegada. Para marcar ese orden hay dos contadores compartidos, uno que indica cual es el número del último coche (last\_car), y otro que indica qué coche es el siguiente en cruzar (first\_car).

Como el semaforo cambia inmediatamente, tanto coches como peatones tienen que asegurarse de que el cruce está libre. Es decir, aunque tengan el semaforo en verde, los coches deberían esperar a que los peatones terminen de cruzar (y viceversa).

---

```
#define CARS 0
#define PEDESTRIANS 1
#define SEMAPHORE_TIME ...

struct crossing {
    int first_car; // Number of the first car that is waiting
    int last_car; // Number of the last car that is waiting
    int green; // Who has the green light right now
    mtx_t cross_m;
    cnd_t cars_wait, peds_wait;
};

int semaphore(void *ptr) {
    struct crossing *cross = ptr;

    while(true) {
        mtx_lock(&cross->cross_m);
        cross->green = (cross->green + 1) % 2;
        if(cross->green == CARS) cnd_broadcast(&cross->cars_wait);
        else cnd_broadcast(&cross->peds_wait);
        mtx_unlock(&cross->cross_m);
        sleep(SEMAPHORE_TIME);
    }
}

int pedestrian(void *ptr) {
    struct crossing *cross = ptr;
    // pedestrian enter

    do_cross(); // simulates the actual crossing.

    // Pedestrian exit
}

int car(void *ptr) {
    struct crossing *cross = ptr;
    // car enters

    do_cross(); // simulates the actual crossing.

    // car exits
}
```

---

Implemente las funciones pedestrian y car. Puede modificar la estructura crossing si lo necesita.

## Solución

---

```
#define CARS 0
#define PEDESTRIANS 1
#define SEMAPHORE_TIME ...

struct crossing {
    int first_car; // Number of the first car that is waiting (starts at 0)
    int last_car; // Number of the last car that is waiting (starts at 0)
    int green; // Who has the green light right now
    mtx_t cross_m;
    int pedestrians; // starts at 0
    int cars; // starts at 0
    cond_t cars_wait, peds_wait;
};

int semaphore(void *ptr) {
    struct crossing *cross = ptr;

    while(true) {
        mtx_lock(&cross->cross_m);
        cross->green = (cross->green + 1) % 2;
        if(cross->green == CARS) cnd_broadcast(&cross->cars_wait);
        else cnd_broadcast(&cross->peds_wait);
        mtx_unlock(&cross->cross_m);
        sleep(SEMAPHORE_TIME);
    }
}

int pedestrian(void *ptr) {
    struct crossing *cross = ptr;

    mtx_lock(cross->cross_m);
    while(cross->green != PEDESTRIANS || cross->cars > 0)
        cnd_wait(&cross->peds_wait, &cross->cross_m);
    cross->pedestrians++;
    mtx_unlock(cross->cross_m);

    do_cross(); // simulates the actual crossing.

    mtx_lock(cross->cross_m);
    cross->pedestrians--;
    if(cross->pedestrians == 0 && cross->green == CARS)
        cnd_broadcast(cross->cars_wait);
    mtx_unlock(cross->cross);

    return 0;
}

int car(void *ptr) {
    struct crossing *cross = ptr;
    int myturn;

    mtx_lock(cross->cross_m);
    myturn = cross->last_car;
    cross->last_car++;
    while(cross->green != CARS || cross->first_car != myturn || cross->pedestrians > 0)
        cnd_wait(&cross->cars_wait, &cross->cross_m);
    cross->cars = 1;
    mtx_unlock(cross->cross_m);

    do_cross(); // simulates the actual crossing.

    mtx_lock(cross->cross_m);
    cross->cars = 0;
    cross->first_car++;
    if(cross->green == CARS) cnd_broadcast(&cross->cars_wait);
    else cnd_broadcast(&cross->peds_wait);
    mtx_unlock(cross->cross_m);

    return 0;
}
```

---